



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Fátima Joanna López Quiñonez **Carné:** 1088825

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Es una forma de comunicar dos sistemas por medio de HTTP, donde todo se trata como un recurso que se manipula con métodos estándar. Cada petición es independiente y lleva toda la información necesaria.

2. ¿Qué es un endpoint?

Es un punto de acceso específico de una API para realizar una operación sobre un recurso.

3. ¿Qué diferencia existe entre una URL y un endpoint?

La URL es la dirección completa de un recurso en la web. El endpoint es esa misma ruta junto con el método HTTP, es decir, con la operación disponible allí.

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

GET consulta datos, POST crea un recurso, PUT lo reemplaza completo, PATCH lo modifica parcialmente y DELETE lo elimina.

5. ¿Qué es JSON y por qué es común en las APIs?

Es un formato de texto ligero que organiza datos en pares clave-valor, como {"nombre": "Ana"}. Es común en las APIs porque es fácil de leer, liviano y compatible con casi todos los lenguajes.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Es un número que indica el resultado de la petición. Ejemplos: 200 (éxito), 404 (recurso no encontrado) y 500 (error interno del servidor).

7. ¿Qué información se puede encontrar en la respuesta de una API?

Incluye el código de estado, los encabezados y el cuerpo con los datos, normalmente en JSON. También puede traer mensajes de error si algo falló.

8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

Permite probar endpoints sin escribir código, lo que ayuda a detectar errores antes de programar. Además, se pueden guardar las peticiones y reutilizarlas.

9. ¿Qué es y para qué sirve Swagger UI?

Es una interfaz web interactiva que muestra la documentación de una API. Permite ver los endpoints y probarlos directamente desde el navegador.

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Es una herramienta para diseñar, probar y documentar APIs enviando peticiones y analizando las respuestas. También permite automatizar pruebas y compartirlas con el equipo.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingrese a Swagger Petstore y localice las operaciones relacionadas con /pet



- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
- ¿Qué datos fueron enviados en el body?
- ¿Qué código HTTP devolvió la API?
- ¿Cómo comprobó que el cambio realmente ocurrió?

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota



- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.